Architectural Specifics

Task A : Implementing RLHF/PPO

1. Reward Model:
   - Initialized `bert-base-uncased` as the base reward model.
   - Implemented the Bradley-Terry loss for reward modelling.
   - Fine-tuned it for 3 epochs on the complete training data.

   Hyperparameters:
   - `learning_rate` = 2e-5
   - `batch_size` = 8
   - `num_epochs` = 3
   - Size of train data = 24000 samples
   - ETA = 15-20 mins
   - GPU Memory = 3 GB

2. RLHF/PPO:

   - Initialized `gpt2-medium` as both the policy and ref_policy models.
   - Initialized `gpt2-small` as the value model.
   - Initialized the fine-tuned reward model (`A1_20CS30049_reward_model.pt`) as the reward model.
   - Huggingface's PPOTrainer intrinsically implements the reward computation as part of the trainer's code.
   - However, it assumes that ***both the reward model and policy/value models have the same architectures!***

   - NOT THE CASE HERE!
     - **Reward Model** : BertForSequenceClassification
     - **Policy Model** : GPT2LMHeadModel (Causal LM)

   - **Architectural Workarounds**:

     - `trl`'s `get_reward()` function requires two properties from the passed reward model:
       - model should have a `base_model_prefix`
       - model should implement a `score()` function to score the reward logits

```
lm_backbone = getattr(model, model.base_model_prefix)
input_ids = torch.masked_fill(query_responses, ~attention_mask, 0)
output = lm_backbone(
    input_ids=input_ids,
    attention_mask=attention_mask,
    position_ids=position_ids,
    return_dict=True,
    output_hidden_states=True,
    use_cache=False,  # otherwise mistral-based RM would error out
)
reward_logits = model.score(output.hidden_states[-1])
```

To implement these functionalities, we define two new classes:

1. **class GPT2BERT**
   - A wrapper class which loads the fine-tuned reward model as `self.bert`,
   - It first converts GPT-tokenizer created input IDs to BERT tokens for compatibility
   - It then re-shapes the BERT-based output hidden states to be compatible with the original GPT-tokenized `input_ids` (So, that reward scoring can be done per GPT token, as TRL's `get_reward` intends)
   - It also implements the `score()` function for TRL's `get_reward()` by using `self.bert.classifier` on the input hidden states.

2. **class RewardModel**

- A wrapper class around GPT2Bert which is the actual object type passed as reward model to HF PPOTrainer.
- It defines the `gpt2bert` wrapper class as its object `self.gpt2bert`.
- It implements the `model.base_model_prefix` as intended.
- It also re-defines `score()` by calling `self.gpt2bert.score()` defined earlier.

- **Hyper-Parameters**:
  - `finetuning_type` : Full Fine-tuning
  - `learning_rate`: 2e-6
  - `train_batch_size`: 4
  - `eval_batch_size`: 2
  - `num_ppo_epochs`: 2
  - `gamma` : 1.0
  - `lam` : 0.95
  - `cliprange` : 0.2
  - `cliprange_value` : 0.2
  - `vf_coef` : 0.1
  - `kl_coef` : 0.05
  - Training Data Size = 5000 (20.8%) (Due to time constraints)
  - ETA = 1.5 hrs
  - GPU Memory = 4 GB

Task B : Implementing DPO

- Initialized `gpt2-medium` as the policy & ref_policy models.
- **Architectural Issues**:
    - GPT-2 computes log_probs for the prompt + response tokens combined!
    - However, summation of log_probs for all prompt and response tokens gives:
        - $P(y, x)$ (JOINT PROBABILITY)
          instead of
        - $P(y|x)$ (CONDITIONAL PROBABILITY)
    - Need to remove all prompt `token_log_probs` contributions per sample.
    - Achieved this by defining a `prompt_nmask`,
    - Defined as the NOT of the prompt's attention mask
    - It has zeroes for all positions occupied by prompt tokens
    - While `compute_logprobs()` is being computed,

```
token_logprobs = token_logprobs * prompt_nmask
final_mask = final_mask * prompt_nmask
```

    The 2nd line zeroes out the prompt token logprobs, as intended.

- **Hyper-parameters**:
    - `finetuning_type` : Full Fine-tuning
    - `learning_rate`: 2e-6
    - `train_batch_size`: 4
    - `num_epochs` : 3
    - `beta` : 0.001
    - Training Data Size = 24000 samples
    - ETA = 3 hrs
    - GPU Memory = 25 GB

Evaluations

Evaluations done on the whole test set (6000 questions)

Response Quality (Higher is better)

BLEU Score

RLHF: 0.0014
DPO: 0.0002

ROUGE-L

RLHF: 0.0828
DPO: 0.0073

A qualitative analysis of sample responses shows that RLHF is able to at least maintain good-quality sample quality after training on the base model. DPO, on the other hand, adversely affects the response quality after full training on the data. Though this might be due to sub-optimal hyper-parameters, I did not manage to obtain qualitative response improvements for 2 learning rate values of `2e-5` and `2e-6`.

Computation Cost (Lower is better)

Average Generation Time (Per inference prompt)

RLHF: 0.8439s
DPO: 1.9391s

DPO, in general, was way more compute-intensive than PPO for the same data scale. For instance, PPO only took **4 GB** of GPU VRAM for a train batch size of 4, while DPO took **~25GB VRAM** for the same scenario. However, PPO also took more time for training (ETA = ~10 hrs) for fine-tuning on the full data, which is why the choice for sampling 20% of data for PPO training was done.

Sample Efficiency Analysis

RLHF Sample Efficiency

BLEU progression

25% data: 0.0011
50% data: 0.0015
75% data: 0.0014
100% data: 0.0014

ROUGE-L progression

25% data: 0.0758
50% data: 0.0819
75% data: 0.0827
100% data: 0.0828

DPO Sample Efficiency

BLEU progression

25% data: 0.0001
50% data: 0.0001
75% data: 0.0001
100% data: 0.0002

ROUGE-L progression

25% data: 0.0073
50% data: 0.0071
75% data: 0.0070
100% data: 0.0073

DPO, in general, affects the general response quality after full-finetuning, which reflects in the overall sample efficiency. Moreover, DPO shows little to no improvement with increase in data, while PPO still shows some improvements after fine-tuning on the data subset.

Comparison Summary:

Response Quality Winner: RLHF
Computation Efficiency Winner: RLHF